

Don't Add or Delete Items from a List While Looping Over It

Adding or deleting items from a list while looping (that is, *iterating*) over it with a `for` or `while` loop will most likely cause bugs. Consider this scenario: you want to iterate over a list of strings that describe items of clothing and ensure that there is an even number of socks by inserting a matching sock each time a sock is found in the list. The task seems straightforward: iterate over the list's strings, and when you find 'sock' in a string, such as 'red sock', append another 'red sock' string to the list.

But this code won't work. It gets caught in an infinite loop, and you'll have to press CTRL-C to interrupt it:

```
>>> clothes = ['skirt', 'red sock']
>>> for clothing in clothes: # Iterate over the list.
...     if 'sock' in clothing: # Find strings with 'sock'.
...         clothes.append(clothing) # Add the sock's pair.
...         print('Added a sock:', clothing) # Inform the user.
...
Added a sock: red sock
Added a sock: red sock
Added a sock: red sock
--snip--
Added a sock: red sock
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
KeyboardInterrupt
```

You'll find a visualization of the execution of this code at <https://autbor.com/addingloop/>.

The problem is that when you append 'red sock' to the `clothes` list, the list now has a new, third item that it must iterate over: ['skirt', 'red sock', 'red sock']. The `for` loop reaches the second 'red sock' on the next iteration, so it appends *another* 'red sock' string. This makes the list ['skirt', 'red sock', 'red sock', 'red sock'], giving the list another string for Python to iterate over. This will continue happening, as shown in [Figure 8-1](#), which is why we see the never-ending stream of 'Added a sock.' messages. The loop only stops